

PART III**ARRAYS AND STRINGS**

The two data shapes behind most problems you will ever solve

Arrays and strings look simple, which is exactly why beginners under-invest in them. This Part treats them as seriously as the rest of the book treats graphs or DP — because a huge fraction of medium and even hard problems are really "an array/string problem wearing a disguise."

Contents of Part III

- 1. Arrays: The Memory Model
- 2. Array Traversal, Search, and In-Place Manipulation
- 3. Vectors — Dynamic Arrays Done Right
- 4. 2D Arrays and Matrices
- 5. Subarrays and Subsequences
- 6. Strings: Fundamentals
- 7. String Comparison, Parsing, and stringstream
- 8. Classic String Problems
- 9. Two Pointers
- 10. Sliding Window
- Part III Revision, Cheat Sheet, and Practice

Chapter 1 — Arrays: The Memory Model

1.1 What an array actually is

In one sentence: an array is a block of memory holding a fixed number of same-typed values, laid out one right after another, so any element can be reached instantly if you know its position.

```
int a[5] = {10, 20, 30, 40, 50};
```

Because every element has the same type and size, the computer can jump straight to element i by computing $\text{base_address} + i \times \text{element_size}$ — no searching required. This is why array access, $a[i]$, is $O(1)$: it's a single arithmetic calculation, not a scan.

1.2 Zero-based indexing

C++ arrays are 0-indexed: the first element is $a[0]$, not $a[1]$. An array of size n has valid indices 0 through $n-1$. $a[n]$ is out of bounds.

WARNING — Out-of-bounds access is not always caught

C++ does not check array bounds at runtime. Reading or writing $a[n]$ (or $a[-1]$) on a fixed-size array is undefined behaviour: it might crash, might silently corrupt a nearby variable, or might appear to work by accident. This is one of the most dangerous classes of bug in C++ precisely because it doesn't always announce itself.

1.3 Static vs. dynamic arrays

	Static array (int a[n])	Dynamic array (vector v)
Size fixed at compile time?	Yes (unless using new[])	No — can grow/shrink at runtime
Can be resized after creation?	No	Yes
Bounds checking available?	No	Only via .at(i), not operator[]
Typical use in this book	Fixed-size buffers, global DP arrays	Almost everything else

NOTE — Default to vector in competitive code

Unless you have a specific reason to use a fixed-size C-style array (e.g. a very large global array sized once at the top of the file), reach for vector by default — it is resizable, integrates with every STL algorithm, and its performance is effectively identical to a raw array once compiled with optimizations.

1.4 Declaring, initializing, and common bugs

```
int a[5]; // uninitialized – contains garbage values!
int b[5] = {0}; // all elements zero-initialized
int c[5] = {1, 2, 3}; // {1, 2, 3, 0, 0} – remaining elements default to 0
int d[] = {1, 2, 3, 4}; // size 4, inferred from the initializer list
```

WARNING — Uninitialized arrays contain garbage

Unlike some languages, C++ does not automatically zero-initialize a local array. `int a[5];` without an initializer contains whatever bytes happened to already be in that memory — reading it before writing to every position is a common source of "works sometimes, fails other times" bugs.

Chapter 2 — Array Traversal, Search, and In-Place Manipulation

2.1 Traversal

```
for (int i = 0; i < n; i++) cout << a[i] << " "; // index-based
for (int x : a) cout << x << " "; // range-based, read-only copy
for (int &x : a) x *= 2; // range-based, modifies in place
```

Use the index-based form when you need the position itself (e.g. to compare `a[i]` with `a[i+1]`); use the range-based reference form (`int &x`) whenever you want to modify elements in place without manual indexing.

2.2 Finding minimum, maximum, and linear search

```
int mn = a[0], mx = a[0];
for (int i = 1; i < n; i++) {
    mn = min(mn, a[i]);
    mx = max(mx, a[i]);
}

// Linear search for a target value
int idx = -1;
for (int i = 0; i < n; i++) {
    if (a[i] == target) { idx = i; break; }
}
```

NOTE — STL shortcuts

`*min_element(a.begin(), a.end())` and `*max_element(a.begin(), a.end())` do the same min/max scan in one call (Part IV covers the full STL algorithm library). Both are still $O(n)$ — the STL call doesn't make it faster, just shorter to write.

2.3 Frequency counting

```
// When values are small and bounded (e.g. 0..99), a plain array is fastest
int freq[100] = {0};
for (int i = 0; i < n; i++) freq[a[i]]++;

// When values are large or unbounded, use a hash map (Part VII covers this fully)
unordered_map<int,int> freqMap;
for (int i = 0; i < n; i++) freqMap[a[i]]++;
```

Dry run — frequency array

a[i] processed	freq array state (only nonzero shown)
3	freq[3]=1
1	freq[1]=1, freq[3]=1
3	freq[1]=1, freq[3]=2

2.4 Reversing an array in place

```
int left = 0, right = n - 1;
while (left < right) {
    swap(a[left], a[right]);
    left++;
    right--;
}
```

This is the two-pointer pattern in its simplest form (Chapter 9 generalizes it): two indices starting at opposite ends, moving toward each other, doing $O(1)$ work per step — $O(n)$ total, $O(1)$ extra space, no separate array needed.

2.5 Rotating an array

Problem: rotate an array left by k positions, in place, using $O(1)$ extra space.

Naive: rotate one step at a time, k times — $O(n \cdot k)$, too slow if k is close to n .

Optimal (the reversal trick): reverse the first k elements, reverse the remaining $n-k$ elements, then reverse the whole array.

```
void reverseRange(vector<int> &a, int l, int r) {
    while (l < r) swap(a[l++], a[r--]);
}
void rotateLeft(vector<int> &a, int k) {
    int n = a.size();
    k %= n;
    reverseRange(a, 0, k - 1);
    reverseRange(a, k, n - 1);
    reverseRange(a, 0, n - 1);
}
```

Dry run: rotate [1,2,3,4,5] left by 2

Step	Array state
Start	[1,2,3,4,5]
Reverse [0,1]	[2,1,3,4,5]
Reverse [2,4]	[2,1,5,4,3]
Reverse [0,4]	[3,4,5,1,2] ← final answer

KEY IDEA — Why this works

Reversing the whole array puts every element in the opposite order of a full rotation. Pre-reversing each of the two segments individually cancels out that extra reversal within each segment, leaving only the rotation itself. This is a good example of a clever observation (Chapter 5, Part II) turning an $O(n \cdot k)$ approach into $O(n)$.

2.6 Removing duplicates from a sorted array (in place)

```
int removeDuplicates(vector<int> &a) {
    if (a.empty()) return 0;
    int writeIdx = 1;
    for (int i = 1; i < (int)a.size(); i++) {
        if (a[i] != a[writeIdx - 1]) {
            a[writeIdx] = a[i];
            writeIdx++;
        }
    }
    return writeIdx; // new logical length
}
```

This is the write-pointer pattern: one index scans every element, a second, slower-moving index marks where the next "kept" element should be written. It appears constantly whenever you need to filter or compact an array in place without extra memory.

Chapter 3 — Vectors: Dynamic Arrays Done Right

3.1 What a vector is, conceptually

A vector is a dynamic array: internally, it owns a block of memory (exactly like a C-style array) but manages growing that block

automatically as you add elements. It tracks two separate numbers — size (how many elements are actually stored) and capacity (how much memory is currently reserved, which may be larger than size).

```
vector<int> v; // empty vector
vector<int> v2(5); // 5 elements, all 0
vector<int> v3(5, 7); // 5 elements, all 7
vector<int> v4 = {1, 2, 3}; // initializer list
```

3.2 push_back, emplace_back, and how growth works

```
vector<int> v;
v.push_back(10);
v.push_back(20);
v.emplace_back(30); // constructs the element in place – avoids a
                     // copy for complex types
```

When a vector's capacity is full and you push another element, it allocates a new, larger block (typically double the old capacity), copies every existing element over, and frees the old block. This is why an individual `push_back` is occasionally slow (when a reallocation happens) but the average cost per push, across many pushes, is still $O(1)$ — this is called amortized $O(1)$.

NOTE — `emplace_back` vs `push_back`

For simple types like `int` there's no practical difference. For a type like `pair<int,int>` or a custom struct, `emplace_back(a, b)` constructs the object directly inside the vector's memory, while `push_back({a, b})` constructs a temporary first and then copies/moves it in — `emplace_back` is usually the slightly cheaper choice.

3.3 `size()` vs. `capacity()`, and `reserve()`

```
vector<int> v;  
v.reserve(1000); // pre-allocate capacity for 1000 elements, size is  
still 0  
for (int i = 0; i < 1000; i++) v.push_back(i); // no reallocation  
happens during this loop
```

If you know in advance roughly how many elements a vector will end up holding, calling `reserve()` up front avoids repeated reallocation-and-copy steps — a common micro-optimization in performance-sensitive competitive code.

3.4 erase and insert, and their real cost

```
v.erase(v.begin() + 2); // removes the element at index 2, shifts  
everything after it left  
v.insert(v.begin() + 1, 99); // inserts 99 at index 1, shifts  
everything after it right
```

WARNING — erase/insert in the middle are $O(n)$, not $O(1)$

Both operations must shift all elements after the affected position, so a single erase or insert in the middle of a vector costs $O(n)$. Doing this inside a loop turns an apparently-simple piece of code into an accidental $O(n^2)$ algorithm — one of the most common hidden-complexity mistakes in competitive code. If you need frequent insertion or removal in the middle, consider a different structure (a linked list — Part VIII — or restructuring the approach to avoid it).

3.5 Vector of vectors, vector of pairs, vector as adjacency list

```
vector<vector<int>> grid(n, vector<int>(m, 0)); // n x m grid, all
zero
vector<pair<int,int>> points; // e.g. (x, y) coordinates
points.push_back({3, 4});

vector<vector<int>> adj(n); // adjacency list for a graph with n
nodes (Part XIII)
adj[0].push_back(1); // edge from node 0 to node 1
```

These three patterns — a 2D vector for grids, a vector of pairs for coordinate/weight data, and a vector of vectors as a graph's adjacency list — are used so often in competitive C++ that recognizing them on sight is worth the small amount of syntax memorization.

KEY IDEA — Chapter 3 quick revision

A vector separately tracks size (elements stored) and capacity (memory reserved); `push_back` is amortized $O(1)$.

`erase/insert` in the middle of a vector cost $O(n)$ each — avoid them inside a loop over n elements.

`reserve()` up front avoids repeated reallocation when the final size is roughly known.

`vector<vector<int>>` is the default representation for grids and graph adjacency lists.

Chapter 4 — 2D Arrays and Matrices

4.1 Declaring and traversing a grid

```
vector<vector<int>> grid(n, vector<int>(m));
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cin >> grid[i][j];

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) cout << grid[i][j] << " ";
    cout << "\n";
}
```

By convention in this book, *i* indexes rows (0 to *n*−1) and *j* indexes columns (0 to *m*−1) — keep this consistent in your own code to avoid swapping rows and columns by accident, a surprisingly common source of bugs.

4.2 The four/eight-direction neighbour pattern

Grid problems (often disguised graph problems — Part XIII) commonly need to visit a cell's neighbours. A pair of direction arrays makes this a clean loop instead of four repeated if-statements:

```
int dr[] = {-1, 1, 0, 0}; // up, down, left, right
int dc[] = {0, 0, -1, 1};
for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr >= 0 && nr < n && nc >= 0 && nc < m) {
        // (nr, nc) is a valid in-bounds neighbour of (r, c)
    }
}
```

WARNING — Always bounds-check before indexing

Computing a neighbour coordinate that falls outside $[0, n)$ or $[0, m)$ and indexing into the grid anyway is undefined behaviour, exactly as with 1D arrays (Section 1.2). The bounds check must come before the access, not after.

4.3 Transposing and rotating a matrix

```
// Transpose (swap rows and columns) an n x n matrix in place
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        swap(grid[i][j], grid[j][i]);
```

Note the inner loop starts at $j = i + 1$, not 0 — swapping (i,j) with (j,i) for every pair once is enough; starting from 0 would swap each pair twice and undo the transpose.

Chapter 5 — Subarrays and Subsequences

5.1 The critical distinction

These two words are used precisely in competitive programming and are not interchangeable — confusing them is a very common way to misread a problem entirely.

Term	Definition	Example (from [1,2,3])
Subarray	A contiguous run of elements	[2,3] is a subarray; [1,3] is not
Subsequence	Elements in the same relative order, not necessarily contiguous	[1,3] is a subsequence
Subset	Any selection of elements, order doesn't matter	{3,1} is a subset, same as {1,3}

KEY IDEA — Rule of thumb

Contiguous → "subarray" or "substring". Order-preserving but gaps allowed → "subsequence". Order doesn't matter at all → "subset". A problem's exact wording changes the correct algorithm completely — a subarray problem is often solvable in $O(n)$ (sliding window, prefix sums), while the corresponding subsequence or subset version is often exponential without further structure (dynamic programming, Part XV).

5.2 Counting subarrays and subsequences

An array of length n has exactly $n(n+1)/2$ subarrays (choose a start and an end index, $\text{start} \leq \text{end}$) and exactly $2^n - 1$ non-empty subsequences (each element is independently included or not). These counts alone often tell you immediately whether an "enumerate every subarray/subsequence" brute force will fit the constraints (Chapter 3, Part II).

Problem 5.1 — Which is it? [Level 0]

A problem asks: "find the maximum sum you can obtain by picking some elements of the array, keeping their original order, such that no two picked elements are adjacent." Is this a subarray, subsequence, or subset problem?

Approach: Subsequence — order is preserved but elements don't need to be contiguous (in fact adjacency is explicitly forbidden). This phrasing is the classic setup for the 'House Robber' dynamic programming pattern (Part XV).

Chapter 6 — Strings: Fundamentals

6.1 What a C++ string is

`std::string` (from `<string>`, included via `<bits/stdc++.h>`) is, conceptually, a dynamic array of `char` — it supports indexing, has a size, and can grow, exactly like a `vector<char>` with extra text-specific operations layered on top.

```
string s = "hello";
cout << s[0]; // 'h'
cout << s.size(); // 5
cout << s.length(); // 5 – identical to size(), just an alias
s[0] = 'H'; // strings are mutable – "Hello"
```

6.2 Traversal and concatenation

```
for (char c : s) cout << c;

string a = "hello", b = "world";
string c = a + " " + b; // "hello world" – + concatenates
a += "!"; // "hello!" – += appends
```

6.3 Substrings and searching within a string

```
string s = "competitive programming";
cout << s.substr(0, 4); // "comp" – starts at index 0, length 4
cout << s.substr(12); // "programming" – from index 12 to the end
cout << s.find("pro"); // 12 – index where "pro" first occurs
cout << s.find("xyz"); // string::npos if not found (a very large constant)
```

WARNING — Always compare find() against string::npos, not -1

find() returns `size_t`, an unsigned type, and "not found" is represented by the special constant `string::npos`, not `-1`. Writing `if (s.find("x") == -1)` compares against a huge unsigned number due to implicit conversion (Part I, Section 3.5) and will never trigger.

`substr(pos, len)` costs $O(\text{len})$ — it copies characters into a new string. Calling it repeatedly inside a loop over a large string is a common accidental- $O(n^2)$ trap, exactly like the vector erase/insert issue in Chapter 3.

6.4 ASCII and character classification

Every `char` is really a small integer (its ASCII code). This lets you do arithmetic on characters directly:

```
char c = 'c';
int position = c - 'a'; // 2 – zero-based position in the alphabet
char upper = c - 'a' + 'A'; // 'C' – manual uppercase conversion

#include <cctype>
isalpha(c); isdigit(c); isupper(c); islower(c); isspace(c);
char u = toupper(c); char l = tolower(c);
```

KEY IDEA — The subtract-a-base-character trick

`c - 'a'` (for lowercase) or `c - '0'` (for a digit character) converts a character into a small usable index or numeric value. This single trick underlies frequency-array counting for strings, digit-to-number parsing, and a large fraction of string-manipulation code in this book.

Chapter 7 — String Comparison, Parsing, and stringstream

7.1 Comparing strings

```
string a = "apple", b = "banana";  
cout << (a == b); // 0 (false)  
cout << (a < b); // 1 (true) – lexicographic (dictionary) comparison
```

String comparison operators compare lexicographically: character by character, using each character's ASCII value, stopping at the first difference (or when one string ends). This is exactly what a dictionary ordering means, and it is what `sort()` uses by default on a `vector<string>`.

7.2 Converting between strings and numbers

```
int n = stoi("42"); // string to int  
long long big = stoll("123456789012"); // string to long long  
double d = stod("3.14"); // string to double  
string s = to_string(42); // int (or other numeric type) to string
```

7.3 Splitting input with stringstream

When a line contains a variable, unknown-in-advance number of whitespace-separated tokens, `stringstream` is the standard way to split it — read the whole line with `getline`, then extract tokens one at a time exactly like reading from `cin`.

```
string line;  
getline(cin, line);  
stringstream ss(line);  
vector<int> values;  
int x;  
while (ss >> x) values.push_back(x); // reads until the stream runs  
out of tokens
```

Chapter 8 — Classic String Problems

8.1 Palindrome check

```
bool isPalindrome(string &s) {  
    int l = 0, r = s.size() - 1;  
    while (l < r) {  
        if (s[l] != s[r]) return false;  
        l++; r--;  
    }  
    return true;  
}
```

Same two-pointer shape as reversing an array (Chapter 2) — this is not a coincidence: checking "is this a palindrome" is exactly "does this equal its own reverse," verified without actually constructing the reverse. $O(n)$ time, $O(1)$ extra space.

8.2 Anagram check

Problem: do two strings contain exactly the same characters, possibly in a different order?

Naive: sort both strings and compare — $O(n \log n)$.

Optimal: build a frequency count of each string's characters and compare the counts — $O(n)$, using the frequency-array pattern from Chapter 2 combined with the ASCII-index trick from Chapter 6.

```
bool isAnagram(string &a, string &b) {  
    if (a.size() != b.size()) return false;  
    int freq[26] = {0};  
    for (char c : a) freq[c - 'a']++;  
    for (char c : b) freq[c - 'a']--;  
    for (int f : freq) if (f != 0) return false;  
    return true;  
}
```

8.3 Longest common prefix of a set of strings

```
string longestCommonPrefix(vector<string> &words) {  
    string prefix = words[0];  
    for (int i = 1; i < (int)words.size(); i++) {  
        int j = 0;  
        while (j < (int)prefix.size() && j < (int)words[i].size() &&  
prefix[j] == words[i][j]) j++;  
        prefix = prefix.substr(0, j);  
        if (prefix.empty()) break;  
    }  
    return prefix;  
}
```

KEY IDEA — Chapter 8 quick revision

Palindrome check reuses the two-pointer shape from array reversal.

Anagram check: compare 26-length frequency counts rather than sorting, for $O(n)$ instead of $O(n \log n)$.

The `c - 'a'` trick is what makes a fixed-size frequency array possible for lowercase-letter strings.

Chapter 9 — Two Pointers

9.1 The general pattern

Two pointers uses two indices moving through a sequence — either from opposite ends toward each other, or both moving forward at possibly different speeds — to solve a problem in $O(n)$ that would otherwise need $O(n^2)$ with nested loops. It requires some structure to work correctly, most often that the array is sorted, or that a specific monotonic property holds.

9.2 Opposite-direction pointers: pair sum in a sorted array

Problem: given a sorted array, determine if two elements sum to a target T .

```
bool hasPairWithSum(vector<int> &a, int T) {
    int l = 0, r = a.size() - 1;
    while (l < r) {
        int sum = a[l] + a[r];
        if (sum == T) return true;
        else if (sum < T) l++; // sum too small: only increasing the
// left pointer can help
        else r--; // sum too large: only decreasing the right
// pointer can help
    }
    return false;
}
```

Why this is correct (the proof idea)

Because the array is sorted, if $a[l] + a[r]$ is too small, $a[l]$ paired with any index less than r is even smaller — so l can safely be skipped forward without ever missing the answer. The symmetric argument justifies moving r backward when the sum is too large. Each step eliminates at least one index from consideration, so the whole scan is $O(n)$.

Dry run: $a = [1,3,5,7,9]$, $T = 12$

l	r	a[l]+a[r]	Action
0 (1)	4 (9)	10	too small → l++
1 (3)	4 (9)	12	found! return true

9.3 Same-direction pointers: removing duplicates (revisited)

The duplicate-removal code in Chapter 2 (Section 2.6) is also a two-pointer pattern — just with both pointers moving in the same direction at different speeds rather than from opposite ends. Recognizing both shapes as "two pointers" (rather than as unrelated tricks) is what lets you apply the technique to a new, unfamiliar problem.

9.4 Recognizing a two-pointer problem

Clue	Likely pattern
Sorted array + "find a pair/triplet with property X"	Opposite-direction two pointers
"Remove/compact in place" on an array	Same-direction (write-pointer) two pointers
"Longest/shortest contiguous run satisfying X"	Sliding window (Chapter 10 — a two-pointer variant)

Problem 9.1 — Three Sum (conceptual) [Level 3]

Given an array, find all triplets that sum to zero. Describe an $O(n^2)$ approach using two pointers as a building block (rather than the $O(n^3)$ brute force of trying every triplet).

Hint 1: Sort the array first. Then fix one element and use two pointers on the remaining range for the other two.

Key observation: After sorting and fixing index i , finding two elements in $a[i+1..n-1]$ that sum to $-a[i]$ is exactly the pair-sum-in-sorted-array problem from Section 9.2.

Complexity: $O(n \log n)$ for the sort, $O(n)$ two-pointer scan for each of the n choices of $i \rightarrow O(n^2)$ total.

Chapter 10 — Sliding Window

10.1 What a sliding window is

A sliding window maintains a contiguous range $[left, right]$ of an array or string and moves its boundaries forward, adding elements as $right$ advances and removing elements as $left$ advances — reusing work already done for the previous window instead of recomputing from scratch. It is the direct generalization of same-direction two pointers to problems about contiguous ranges with a changing size.

10.2 Fixed-size window: maximum sum of any k consecutive elements

Naive: for each starting position, sum the next k elements — $O(n \cdot k)$.

Optimal: compute the first window's sum directly; for every subsequent window, add the new element entering on the right and subtract the element leaving on the left — $O(n)$.

```
long long maxSumWindowK(vector<int> &a, int k) {
    long long windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += a[i];
    long long best = windowSum;
    for (int i = k; i < (int)a.size(); i++) {
        windowSum += a[i] - a[i - k]; // slide: add new, remove old
        best = max(best, windowSum);
    }
    return best;
}
```

Dry run: a = [2,1,5,1,3,2], k = 3

Window	windowSum	best
[2,1,5]	8	8
[1,5,1]	8-2+1=7	8
[5,1,3]	7-1+3=9	9
[1,3,2]	9-5+2=6	9

10.3 Variable-size window: longest substring with at most k distinct characters

Some sliding-window problems don't have a fixed size — the window grows while a condition holds and shrinks from the left when it's violated. The window never resets to empty and re-scans; left only ever moves forward, which is what keeps the whole process $O(n)$ rather than $O(n^2)$.

```
int longestKDistinct(string &s, int k) {
    unordered_map<char,int> freq;
    int left = 0, best = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        freq[s[right]]++;
        while ((int)freq.size() > k) {
            freq[s[left]]--;
            if (freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }
        best = max(best, right - left + 1);
    }
    return best;
}
```

WARNING — Why this is still $O(n)$, not $O(n^2)$

It looks like there's a loop inside a loop, but the inner while loop only ever advances left, and left can advance at most n times in total across the entire run of the algorithm — not n times per outer iteration. Summed over the whole function, right and left together take at most $2n$ steps: $O(n)$ overall. This "total movement is bounded" argument is the standard way to justify a sliding window's complexity.

10.4 Recognizing a sliding-window problem

- "Longest/shortest/maximum/minimum contiguous subarray or substring satisfying some condition."
- A fixed window size k is given directly ("every k consecutive elements").
- The condition can be checked/maintained incrementally as elements enter or leave the window (a sum, a count, a set of distinct characters) rather than needing to look at the whole window from scratch each time.

KEY IDEA — Chapter 10 quick revision

Fixed window: slide by adding the new right element and removing the old left element — $O(n)$.

Variable window: grow right greedily, shrink left only while a condition is violated; left only ever moves forward.

Justify the complexity by bounding total pointer movement across the whole run, not per iteration.

Problem 10.1 — Longest substring without repeating characters [Level 2]

Given a string, find the length of the longest substring with no repeated characters.

Hint 1: This is a variable-size window: grow right, and shrink left whenever the newly added character already exists in the current window.

Complexity: $O(n)$ time using a frequency set/map, $O(\min(n, \text{alphabet size}))$ extra space.

Part III — Master Revision

Must Remember

- Array indexing is $O(1)$ because position translates directly into a memory address — no scanning.
- C++ never bounds-checks operator[] on arrays, vectors, or strings — an out-of-bounds access is undefined behaviour, not a guaranteed crash.
- vector's `push_back` is amortized $O(1)$; `erase/insert` in the middle are $O(n)$ each — avoid the latter inside a loop over n elements.
- Subarray = contiguous. Subsequence = order preserved, gaps allowed. Subset = order doesn't matter. These are not interchangeable in a problem statement.
- The `c - 'a' / c - '0'` trick converts a character into a usable small index — the basis of fixed-size frequency arrays for text.
- Two pointers turn many $O(n^2)$ pair/range problems into $O(n)$, provided some monotonic structure (usually sortedness) justifies moving a pointer without missing the answer.
- A sliding window's complexity is justified by bounding total pointer movement across the whole run, not by looking at the nested-loop shape alone.

Common Mistakes Recap

Mistake	Fix
Reading an uninitialized C-style array	Always initialize, or use vector which zero-initializes on construction
Comparing <code>string::find()</code> result to -1	Compare against <code>string::npos</code>
Calling <code>substr()</code> repeatedly inside a loop over a large string	Recognize the hidden $O(n)$ cost per call — restructure to avoid repeated <code>substr()</code> where possible
<code>erase()/insert()</code> inside a loop over a vector	Recognize the hidden $O(n)$ cost per call — this silently turns an $O(n)$ loop into $O(n^2)$
Confusing subarray, subsequence, and subset in a problem statement	Re-read the exact wording before choosing an algorithm
Transposing a matrix by looping j from 0 instead of $i+1$	Start the inner loop at $j = i + 1$ to avoid double-swapping

Complexity Table — Part III

Operation	Complexity
Array/vector/string index access <code>a[i]</code>	$O(1)$
Linear search / min / max scan	$O(n)$
vector <code>push_back</code> (amortized)	$O(1)$ amortized
vector <code>erase/insert</code> in the middle	$O(n)$
string <code>substr(pos, len)</code>	$O(len)$
Two pointers over an array of size n	$O(n)$
Sliding window over an array/string of size n	$O(n)$
Enumerating all subarrays	$O(n^2)$ of them
Enumerating all subsequences	$O(2^n)$ of them

Interview-Style Questions

- 1 Why is array indexing $O(1)$ but `erase()` from the middle of a vector $O(n)$?
- 2 What is the difference between `size()` and `capacity()` for a vector, and why does that distinction exist?
- 3 Why must you compare `string::find()`'s result against `string::npos` rather than `-1`?
- 4 Explain why a sliding window with an inner while loop is still $O(n)$ overall rather than $O(n^2)$.
- 5 Give a problem statement where the answer differs depending on whether it asks for a subarray or a subsequence.

Cheat Sheet — Part III

```
// Two pointers (opposite ends)
int l = 0, r = n - 1;
while (l < r) { /* adjust l and/or r based on a condition */ }

// Fixed-size sliding window
for (int i = 0; i < k; i++) windowSum += a[i];
for (int i = k; i < n; i++) windowSum += a[i] - a[i-k];

// Variable-size sliding window
int left = 0;
for (int right = 0; right < n; right++) {
    // add a[right] to window state
    while (/* window invalid */) { /* remove a[left] from state */
        left++;
    }
    // update answer using [left, right]
}

// Frequency array for lowercase text
int freq[26] = {0};
for (char c : s) freq[c - 'a']++;
```

NOTE — Coming up in Part IV

Part IV — C++ STL goes deep on every container and algorithm used casually throughout this Part — vector, pair, map, set, stack, queue, priority_queue, and the sort/search/accumulate family — including their real complexity guarantees and the competitive-programming combinations (vector<>, map<int>, custom comparators) that show up in almost every non-trivial solution from here on.